To develop a better strategy, note that the Newton step (9.7.3) is a *descent direction* for f:

$$\nabla f \cdot \delta \mathbf{x} = (\mathbf{F} \cdot \mathbf{J}) \cdot (-\mathbf{J}^{-1} \cdot \mathbf{F}) = -\mathbf{F} \cdot \mathbf{F} < 0 \tag{9.7.5}$$

Thus our strategy is quite simple: We always first try the full Newton step, because once we are close enough to the solution we will get quadratic convergence. However, we check at each iteration that the proposed step reduces f. If not, we *backtrack* along the Newton direction until we have an acceptable step. Because the Newton step is a descent direction for f, we are guaranteed to find an acceptable step by backtracking. We will discuss the backtracking algorithm in more detail below.

Note that this method minimizes f only "incidentally," either by taking Newton steps designed to bring $\mathbf{F}$ to zero, or by backtracking along such a step. The method is *not* equivalent to minimizing f directly by taking Newton steps designed to bring ∇f to zero. While the method can nevertheless still fail by converging to a local minimum of f that is not a root (as in Figure 9.6.1), this is quite rare in real applications. The routine `newt` below will warn you if this happens. The only remedy is to try a new starting point.

9.7.1 Line Searches and Backtracking

When we are not close enough to the minimum of f, taking the full Newton step $\mathbf{p} = \delta \mathbf{x}$ need not decrease the function; we may move too far for the quadratic approximation to be valid. All we are guaranteed is that *initially* f decreases as we move in the Newton direction. So the goal is to move to a new point $\mathbf{x}_{\text{new}}$ along the *direction* of the Newton step $\mathbf{p}$, but not necessarily all the way:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \lambda \mathbf{p}, \qquad 0 < \lambda \leq 1 \tag{9.7.6}$$

The aim is to find λ so that $f(\mathbf{x}_{\text{old}} + \lambda \mathbf{p})$ has decreased sufficiently. Until the early 1970s, standard practice was to choose λ so that $\mathbf{x}_{\text{new}}$ exactly minimizes f in the direction $\mathbf{p}$. However, we now know that it is extremely wasteful of function evaluations to do so. A better strategy is as follows: Since $\mathbf{p}$ is always the Newton direction in our algorithms, we first try $\lambda = 1$, the full Newton step. This will lead to quadratic convergence when $\mathbf{x}$ is sufficiently close to the solution. However, if $f(\mathbf{x}_{\text{new}})$ does not meet our acceptance criteria, we *backtrack* along the Newton direction, trying a smaller value of λ, until we find a suitable point. Since the Newton direction is a descent direction, we are guaranteed to decrease f for sufficiently small λ.

What should the criterion for accepting a step be? It is *not* sufficient to require merely that $f(\mathbf{x}_{\text{new}}) < f(\mathbf{x}_{\text{old}})$. This criterion can fail to converge to a minimum of f in one of two ways. First, it is possible to construct a sequence of steps satisfying this criterion with f decreasing too slowly relative to the step lengths. Second, one can have a sequence where the step lengths are too small relative to the initial rate of decrease of f. (For examples of such sequences, see [2], p. 117.)

A simple way to fix the first problem is to require the *average* rate of decrease of f to be at least some fraction α of the *initial* rate of decrease $\nabla f \cdot \mathbf{p}$:

$$f(\mathbf{x}_{\text{new}}) \leq f(\mathbf{x}_{\text{old}}) + \alpha \nabla f \cdot (\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{old}}) \tag{9.7.7}$$

Here the parameter α satisfies $0 < \alpha < 1$. We can get away with quite small values of α; $\alpha = 10^{-4}$ is a good choice.

The second problem can be fixed by requiring the rate of decrease of f at $\mathbf{x}_{\text{new}}$ to be greater than some fraction β of the rate of decrease of f at $\mathbf{x}_{\text{old}}$. In practice, we will not need to impose this second constraint because our backtracking algorithm will have a built-in cutoff to avoid taking steps that are too small.

Here is the strategy for a practical backtracking routine: Define

$$g(\lambda) \equiv f(\mathbf{x}_{\text{old}} + \lambda \mathbf{p}) \tag{9.7.8}$$

so that

$$g'(\lambda) = \nabla f \cdot \mathbf{p} \tag{9.7.9}$$

If we need to backtrack, then we model g with the most current information we have and choose λ to minimize the model. We start with $g(0)$ and $g'(0)$ available. The first step is always the Newton step, $\lambda = 1$. If this step is not acceptable, we have available $g(1)$ as well. We can therefore model $g(\lambda)$ as a quadratic:

$$g(\lambda) \approx [g(1) - g(0) - g'(0)]\lambda^2 + g'(0)\lambda + g(0) \tag{9.7.10}$$

Taking the derivative of this quadratic, we find that it is a minimum when

$$\lambda = -\frac{g'(0)}{2[g(1) - g(0) - g'(0)]} \tag{9.7.11}$$

Since the Newton step failed, one can show that $\lambda \lesssim \frac{1}{2}$ for small α. We need to guard against too small a value of λ, however. We set $\lambda_{\min} = 0.1$.

On second and subsequent backtracks, we model g as a cubic in λ, using the previous value $g(\lambda_1)$ and the second most recent value $g(\lambda_2)$:

$$g(\lambda) = a\lambda^3 + b\lambda^2 + g'(0)\lambda + g(0) \tag{9.7.12}$$

Requiring this expression to give the correct values of g at λ_1 and λ_2 gives two equations that can be solved for the coefficients a and b:

$$\begin{bmatrix} a \\ b \end{bmatrix} = \frac{1}{\lambda_1 - \lambda_2} \begin{bmatrix} 1/\lambda_1^2 & -1/\lambda_2^2 \\ -\lambda_2/\lambda_1^2 & \lambda_1/\lambda_2^2 \end{bmatrix} \cdot \begin{bmatrix} g(\lambda_1) - g'(0)\lambda_1 - g(0) \\ g(\lambda_2) - g'(0)\lambda_2 - g(0) \end{bmatrix} \tag{9.7.13}$$

The minimum of the cubic (9.7.12) is at

$$\lambda = \frac{-b + \sqrt{b^2 - 3ag'(0)}}{3a} \tag{9.7.14}$$

We enforce that λ lie between $\lambda_{\max} = 0.5\lambda_1$ and $\lambda_{\min} = 0.1\lambda_1$.

The routine has two additional features, a minimum step length `alamin` and a maximum step length `stpmax`. `lnsrch` will also be used in the quasi-Newton minimization routine `dfpmin` in the next section.

roots_multidim.h

```
template <class T>
void lnsrch(VecDoub_I &xold, const Doub fold, VecDoub_I &g, VecDoub_IO &p,
VecDoub_O &x, Doub &f, const Doub stpmax, Bool &check, T &func) {
```

Given an `n`-dimensional point `xold[0..n-1]`, the value of the function and gradient there, `fold` and `g[0..n-1]`, and a direction `p[0..n-1]`, finds a new point `x[0..n-1]` along the direction `p` from `xold` where the function or functor `func` has decreased "sufficiently." The new function value is returned in `f`. `stpmax` is an input quantity that limits the length of the steps so that you do not try to evaluate the function in regions where it is undefined or subject to overflow. `p` is usually the Newton direction. The output quantity `check` is `false` on a normal exit. It is `true` when `x` is too close to `xold`. In a minimization algorithm, this usually signals convergence and can be ignored. However, in a zero-finding algorithm the calling program should check whether the convergence is spurious.

```
    const Doub ALF=1.0e-4, TOLX=numeric_limits<Doub>::epsilon();
```

`ALF` ensures sufficient decrease in function value; `TOLX` is the convergence criterion on Δx.

```
    Doub a,alam,alam2=0.0,alamin,b,disc,f2=0.0;
    Doub rhs1,rhs2,slope=0.0,sum=0.0,temp,test,tmplam;
    Int i,n=xold.size();
    check=false;
    for (i=0;i<n;i++) sum += p[i]*p[i];
    sum=sqrt(sum);
    if (sum > stpmax)
        for (i=0;i<n;i++)
```

```
            p[i] *= stpmax/sum;                    Scale if attempted step is too big.
        for (i=0;i<n;i++)
            slope += g[i]*p[i];
        if (slope >= 0.0) throw("Roundoff problem in lnsrch.");
        test=0.0;                                  Compute λmin.
        for (i=0;i<n;i++) {
            temp=abs(p[i])/MAX(abs(xold[i]),1.0);
            if (temp > test) test=temp;
        }
        alamin=TOLX/test;
        alam=1.0;                                  Always try full Newton step first.
        for (;;) {                                 Start of iteration loop.
            for (i=0;i<n;i++) x[i]=xold[i]+alam*p[i];
            f=func(x);
            if (alam < alamin) {                   Convergence on Δx. For zero finding, the calling program should verify the convergence.
                for (i=0;i<n;i++) x[i]=xold[i];
                check=true;
                return;
            } else if (f <= fold+ALF*alam*slope) return;    Sufficient function decrease.
            else {                                 Backtrack.
                if (alam == 1.0)
                    tmplam = -slope/(2.0*(f-fold-slope));   First time.
                else {                             Subsequent backtracks.
                    rhs1=f-fold-alam*slope;
                    rhs2=f2-fold-alam2*slope;
                    a=(rhs1/(alam*alam)-rhs2/(alam2*alam2))/(alam-alam2);
                    b=(-alam2*rhs1/(alam*alam)+alam*rhs2/(alam2*alam2))/(alam-alam2);
                    if (a == 0.0) tmplam = -slope/(2.0*b);
                    else {
                        disc=b*b-3.0*a*slope;
                        if (disc < 0.0) tmplam=0.5*alam;
                        else if (b <= 0.0) tmplam=(-b+sqrt(disc))/(3.0*a);
                        else tmplam=-slope/(b+sqrt(disc));
                    }
                    if (tmplam>0.5*alam)
                        tmplam=0.5*alam;           λ ≤ 0.5λ1.
                }
            }
            alam2=alam;
            f2 = f;
            alam=MAX(tmplam,0.1*alam);             λ ≥ 0.1λ1.
        }                                          Try again.
    }
```

9.7.2 Globally Convergent Newton Method

Using the above results on backtracking, here is the globally convergent Newton routine `newt` that uses `lnsrch`. A feature of `newt` is that you need not supply the Jacobian matrix analytically; the routine will attempt to compute the necessary partial derivatives of $\mathbf{F}$ by finite differences in the routine `NRfdjac`. This routine uses some of the techniques described in §5.7 for computing numerical derivatives. Of course, you can always replace `NRfdjac` with a routine that calculates the Jacobian analytically if this is easy for you to do.

The routine requires a user-supplied function or functor that computes the vector of functions to be zeroed. Its declaration as a function is

```
VecDoub vecfunc(VecDoub_I x);
```

(The name `vecfunc` is arbitrary.) The declaration as a functor is similar.

roots_multidim.h

```
template <class T>
void newt(VecDoub_IO &x, Bool &check, T &vecfunc) {
```

Given an initial guess `x[0..n-1]` for a root in `n` dimensions, find the root by a globally convergent Newton's method. The vector of functions to be zeroed, called `fvec[0..n-1]` in the routine below, is returned by the user-supplied function or functor `vecfunc` (see text). The output quantity `check` is `false` on a normal return and `true` if the routine has converged to a local minimum of the function `fmin` defined below. In this case try restarting from a different initial guess.

```
    const Int MAXITS=200;
    const Doub TOLF=1.0e-8,TOLMIN=1.0e-12,STPMX=100.0;
    const Doub TOLX=numeric_limits<Doub>::epsilon();
```

Here `MAXITS` is the maximum number of iterations; `TOLF` sets the convergence criterion on function values; `TOLMIN` sets the criterion for deciding whether spurious convergence to a minimum of `fmin` has occurred; `STPMX` is the scaled maximum step length allowed in line searches; and `TOLX` is the convergence criterion on $\delta\mathbf{x}$.

```
    Int i,j,its,n=x.size();
    Doub den,f,fold,stpmax,sum,temp,test;
    VecDoub g(n),p(n),xold(n);
    MatDoub fjac(n,n);
    NRfmin<T> fmin(vecfunc);                    Set up NRfmin object.
    NRfdjac<T> fdjac(vecfunc);                  Set up NRfdjac object.
    VecDoub &fvec=fmin.fvec;                    Make an alias to simplify coding.
    f=fmin(x);                                  fvec is also computed by this call.
    test=0.0;                                   Test for initial guess being a root. Use
    for (i=0;i<n;i++)                               more stringent test than simply TOLF.
        if (abs(fvec[i]) > test) test=abs(fvec[i]);
    if (test < 0.01*TOLF) {
        check=false;
        return;
    }
    sum=0.0;
    for (i=0;i<n;i++) sum += SQR(x[i]);         Calculate stpmax for line searches.
    stpmax=STPMX*MAX(sqrt(sum),Doub(n));
    for (its=0;its<MAXITS;its++) {              Start of iteration loop.
        fjac=fdjac(x,fvec);
```

If analytic Jacobian is available, you can replace the struct `NRfdjac` below with your own struct.

```
        for (i=0;i<n;i++) {                     Compute ∇f for the line search.
            sum=0.0;
            for (j=0;j<n;j++) sum += fjac[j][i]*fvec[j];
            g[i]=sum;
        }
        for (i=0;i<n;i++) xold[i]=x[i];         Store x,
        fold=f;                                 and f.
        for (i=0;i<n;i++) p[i] = -fvec[i];      Right-hand side for linear equations.
        LUdcmp alu(fjac);                       Solve linear equations by LU decompo-
        alu.solve(p,p);                             sition.
        lnsrch(xold,fold,g,p,x,f,stpmax,check,fmin);
```

`lnsrch` returns new $\mathbf{x}$ and f. It also calculates `fvec` at the new $\mathbf{x}$ when it calls `fmin`.

```
        test=0.0;                               Test for convergence on function values.
        for (i=0;i<n;i++)
            if (abs(fvec[i]) > test) test=abs(fvec[i]);
        if (test < TOLF) {
            check=false;
            return;
        }
        if (check) {                            Check for gradient of f zero, i.e., spu-
            test=0.0;                               rious convergence.
            den=MAX(f,0.5*n);
            for (i=0;i<n;i++) {
                temp=abs(g[i])*MAX(abs(x[i]),1.0)/den;
                if (temp > test) test=temp;
            }
```

```
            check=(test < TOLMIN);
            return;
        }
        test=0.0;                                   Test for convergence on δx.
        for (i=0;i<n;i++) {
            temp=(abs(x[i]-xold[i]))/MAX(abs(x[i]),1.0);
            if (temp > test) test=temp;
        }
        if (test < TOLX)
            return;
    }
    throw("MAXITS exceeded in newt");
}
```

roots_multidim.h

```
template <class T>
struct NRfdjac {
```

Computes forward-difference approximation to Jacobian.

```
    const Doub EPS;                 Set to approximate square root of the machine pre-
    T &func;                            cision.
    NRfdjac(T &funcc) : EPS(1.0e-8),func(funcc) {}
```

Initialize with user-supplied function or functor that returns the vector of functions to be zeroed.

```
    MatDoub operator() (VecDoub_I &x, VecDoub_I &fvec) {
```

Returns the Jacobian array `df[0..n-1][0..n-1]`. On input, `x[0..n-1]` is the point at which the Jacobian is to be evaluated and `fvec[0..n-1]` is the vector of function values at the point.

```
        Int n=x.size();
        MatDoub df(n,n);
        VecDoub xh=x;
        for (Int j=0;j<n;j++) {
            Doub temp=xh[j];
            Doub h=EPS*abs(temp);
            if (h == 0.0) h=EPS;
            xh[j]=temp+h;           Trick to reduce finite-precision error.
            h=xh[j]-temp;
            VecDoub f=func(xh);
            xh[j]=temp;
            for (Int i=0;i<n;i++)   Forward difference formula.
                df[i][j]=(f[i]-fvec[i])/h;
        }
        return df;
    }
};
```

roots_multidim.h

```
template <class T>
struct NRfmin {
```

Returns $f = \frac{1}{2}\mathbf{F}\cdot\mathbf{F}$. Also stores value of $\mathbf{F}$ in `fvec`.

```
    VecDoub fvec;
    T &func;
    Int n;
    NRfmin(T &funcc) : func(funcc){}
```

Initialize with user-supplied function or functor that returns the vector of functions to be zeroed.

```
    Doub operator() (VecDoub_I &x) {
```

Returns f at `x`, and stores $\mathbf{F}(\mathbf{x})$ in `fvec`.

```
        n=x.size();
        Doub sum=0;
        fvec=func(x);
        for (Int i=0;i<n;i++) sum += SQR(fvec[i]);
        return 0.5*sum;
    }
};
```